

Introduction to Structured Programming KU

Session 3: Structs & Dynamic Memory

Sebastian David Albu

Summer Semester 2026

Institute of Software Engineering and Artificial Intelligence

LaTeX Template provided by the Institute of Information Security

Roadmap: Intermediate Concepts

Function
Pointers

Structs &
Enums

Dynamic Memory
(Heap)

File I/O

Goal

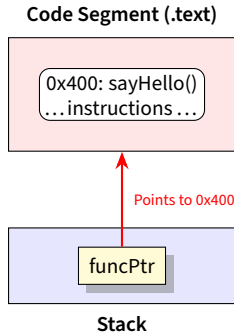
Moving from "Basic Syntax" to "Building Systems". Today we learn how to manage memory manually and make code polymorphic.

Function Pointers

What is a Function Pointer?

Variables store data. Function Pointers store the **address of code**.

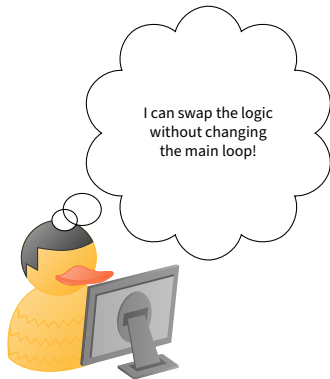
```
1 #include <stdio.h>
2
3 void sayHello()
4 {
5     printf("Hi!\n");
6 }
7
8 int main()
9 {
10     // 1. Declare
11     void (*funcPtr)();
12     // 2. Assign (Point to code)
13     funcPtr = sayHello;
14     // 3. Call
15     funcPtr();
16     return 0;
17 }
18
```



Why use them? (Polymorphism)

We can switch behavior at runtime (like selecting a strategy).

```
1 int add(int a, int b) { return a + b; }
2 int mul(int a, int b) { return a * b; }
3
4 int main()
5 {
6     // Pointer definition
7     int (*op)(int, int);
8
9     op = add;
10    printf("Add: %d\n", op(3, 4)); // 7
11
12    op = mul;
13    printf("Mul: %d\n", op(3, 4)); // 12
14    return 0;
15 }
16
```

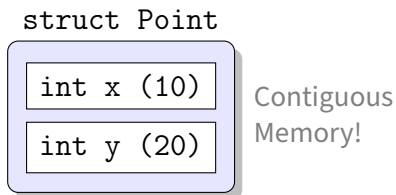


Structs

Structs: Grouping Data

Arrays group data of the *same* type. Structs group data of *different* types.

```
1 struct Point {  
2     int x;  
3     int y;  
4 };  
5  
6 int main()  
7 {  
8     // Initialization  
9     struct Point p1 = {10, 20};  
10    // Access (. operator)  
11    printf("%d", p1.x);  
12 }  
13
```



Passing Structs: Value vs. Reference



```
1 // 1. Pass by Value (Copy) -> Slow
   for big structs
2 void print(struct Point p)
3 {
4     printf("%d", p.x);
5 }
6
7 // 2. Pass by Reference (Pointer) ->
   Fast!
8 void move(struct Point *p)
9 {
10    // Arrow operator ->
11    p->x += 10;
12 }
```

p->x



is Syntactic Sugar for:

(*p).x

The Power of typedef

Writing `struct Point` every time is annoying. `typedef` creates a new name (alias) for a type.

✗ Without typedef

```
1 struct Point
2 {
3     int x;
4     int y;
5 };
6
7 int main()
8 {
9     // Must write "struct"
10    struct Point p1;
11    p1.x = 10;
12 }
13
```

✓ With typedef

```
1 typedef struct Point
2 {
3     int x;
4     int y;
5 } Point_t; // New Name
6
7 int main()
8 {
9     // Cleaner syntax!
10    Point_t p1;
11    p1.x = 10;
12 }
13
```

Syntax Rule: `typedef OldType NewName;`

Pitfall: Struct Initialization

Compiler Error

A struct definition is just a **blueprint** (like a class). You cannot assign values inside the definition because no memory exists yet!

✗ Wrong (Blueprint)

```
1 struct Dog {  
2     // ERROR  
3     int age = 5;  
4     char *name = "Rex";  
5 };  
6
```

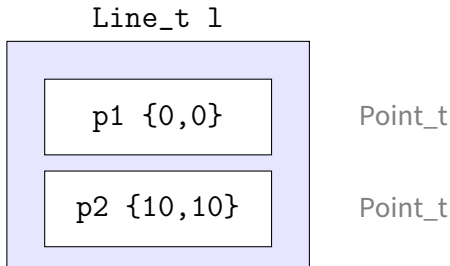
✓ Correct (Instance)

```
1 struct Dog {  
2     int age;  
3     char *name;  
4 };  
5 int main() {  
6     // Initialize the INSTANCE  
7     struct Dog myDog = {5, "Rex"};  
8 }  
9
```

Nested Structs (Composition)

Structs can contain other structs. This allows you to build complex data types.

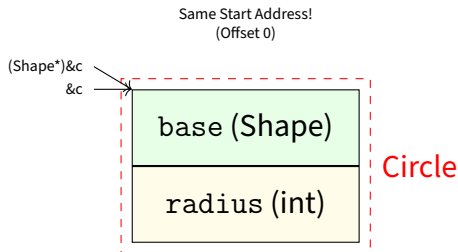
```
1 typedef struct {
2     int x;
3     int y;
4 } Point_t;
5
6 typedef struct {
7     Point_t p1; // Nested
8     Point_t p2; // Nested
9 } Line_t;
10
11 int main() {
12     Point_t a = {0, 0};
13     Point_t b = {10, 10};
14
15     // Line contains two Points
16     Line_t l = {a, b};
17     // Access: l.p1.x
18 }
19
```



Polymorphism in C (Advanced)

The Trick: If a struct is the **first member** of another struct, their memory addresses are identical. We can simulate inheritance!

```
1 typedef struct { char *name; } Shape;
2
3 typedef struct {
4     Shape base; // MUST be first
5     int radius;
6 } Circle;
7
8 void print_shape(Shape *s) {
9     printf("Shape: %s\n", s->name);
10 }
11
12 int main() {
13     Circle c = { {"Round"}, 5 };
14
15     // Treat Circle as Shape!
16     print_shape((Shape*)&c);
17 }
```



Enums

Enums: Magic Numbers Begone

Enums assign names to integer constants. Easier to read than 0, 1, 2.

```
1 enum Level
2 {
3     LOW,          // 0
4     MEDIUM,      // 1
5     HIGH = 10,    // Custom
6     EXTREME       // 11
7 };
8
9 enum Level val = MEDIUM;
```

Name	Value
LOW	0
MEDIUM	1
HIGH	10
EXTREME	11

Why Enums? (vs. #define)



Option A: The Preprocessor

```
1 #define RED 0
2 #define GREEN 1
3 #define BLUE 2
4
5 int color = RED;
6
```

✗ Drawbacks:

- No type information.
- Debuggers just see "0", not "RED".
- Global scope pollution.

Option B: Enums

```
1 enum Color
2 {
3     RED,
4     GREEN,
5     BLUE
6 };
7 enum Color c = RED;
8
```

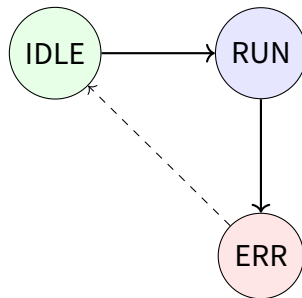
✓ Benefits:

- Logical grouping.
- Debuggers show "RED".
- Code is self-documenting.

The Use Case: State Machines

Enums shine when controlling program logic using switch statements.

```
1 typedef enum {  
2     IDLE,  
3     RUNNING,  
4     ERROR  
5 } State;  
6  
7 void handle_state(State s) {  
8     switch(s) {  
9         case IDLE:  
10            printf("Waiting...");  
11            break;  
12        case RUNNING:  
13            printf("Working...");  
14            break;  
15        case ERROR:  
16            printf("Abort!");  
17            break;  
18    }  
19 }
```



Warning: Enums are just int

In C, Enums are **not strongly typed**. The compiler treats them as integers.

```
1 enum Day { MON, TUE, WED };
2
3 int main() {
4     enum Day today = MON;
5
6     // This is valid C code!
7     // Compiler might not even warn you
8     .
9     today = 1000;
10
11    // You can do math on them
12    today = MON + 5;
13 }
```

C Philosophy:

"Trust the Programmer"
(Even if they make mistakes)



Dynamic Memory

Dynamic Memory

“Why do I hear boss music?”

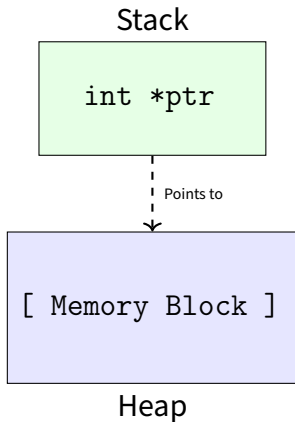
Stack vs. Heap

The Stack (Automatic)

- Small, temporary.
- Variables die at }.
- `int x = 5;`

The Heap (Dynamic)

- Large, manual.
- You control lifetime.
- `malloc / free`

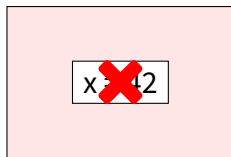


Why do we need the Heap?

The Stack's Limitation: Variables are destroyed when the function returns.

```
1 int* create_int()  
2 {  
3     int x = 42;  
4     return &x;  
5     // DANGER! 'x' dies here.  
6 }  
7  
8 int main()  
9 {  
10    int *p = create_int();  
11    // *p is now garbage!  
12 }  
13
```

Stack Frame (create_int)



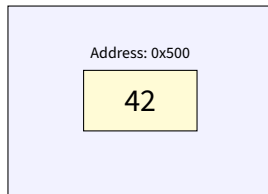
Popped/Destroyed

The Solution: malloc (Manual Allocation)

The **Heap** is a giant pool of free memory. It persists until you manually free it.

```
1 int* create_int()  
2 {  
3     // 1. Ask for 4 bytes  
4     int *p = malloc(sizeof(int));  
5  
6     // 2. Write to that space  
7     *p = 42;  
8  
9     // 3. Return the ADDRESS  
10    return p;  
11 }  
12
```

The Heap (Permanent)



✓ Survives return!

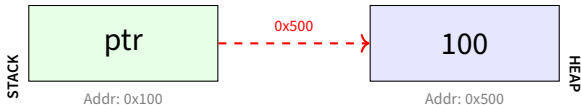
The Bridge: Stack Pointer → Heap Data

Mental Model

The **Pointer** lives on the Stack (temporary).
The **Data** lives on the Heap (permanent).

```
1 void func() {  
2     int *ptr;  
3     ptr = malloc(4);  
4     *ptr = 100;  
5 }  
6
```

If `func()` ends without `free()`, `ptr` dies, but
100 stays forever (**Leak!**).



The Big Three: malloc, calloc, realloc

1. malloc(size)

- Allocates raw bytes. **Content is garbage!**
- `int *p = malloc(5 * sizeof(int));`

2. calloc(count, size)

- Allocates AND **clears to zero**.
- Slightly slower, but safer.

3. realloc(ptr, new_size)

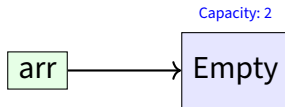
- Resizes an existing block.
- *Warning:* Might move the block to a new address!

Dynamic Array: Step 1 (Allocation)

We want to store numbers, but we don't know how many. We start with a small **Capacity**.



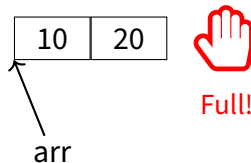
```
1 int main()
2 {
3     // 1. Start small (2 slots)
4     int capacity = 2;
5     int count = 0;
6
7     int *arr = malloc(capacity * sizeof(int));
8     // Always check!
9     if (arr == NULL) return 1;
10
11     // ...
12 }
```



Dynamic Array: Step 2 (The Limit)

We read input until the array is full.

```
1 // ... inside a loop ...  
2  
3 // Store input  
4 arr[count] = 10;  
5 count++; // 1  
6  
7 arr[count] = 20;  
8 count++; // 2  
9  
10 // PROBLEM: Next insert will crash!  
11 // count (2) == capacity (2)  
12
```



Dynamic Array: Step 3 (Expansion)

We double the size using `realloc`.

```
1  if (count == capacity) {
2      capacity *= 2; // New size: 4
3
4      // Use a TEMP pointer for safety!
5      int *temp = realloc(arr, capacity * sizeof(int));
6
7      if (temp == NULL) {
8          // arr is still valid here!
9          free(arr);
10         return 1;
11     }
12     arr = temp; // Update original pointer
13 }
14 arr[count++] = 30; // Safe now!
15
```



Dynamic Array Example

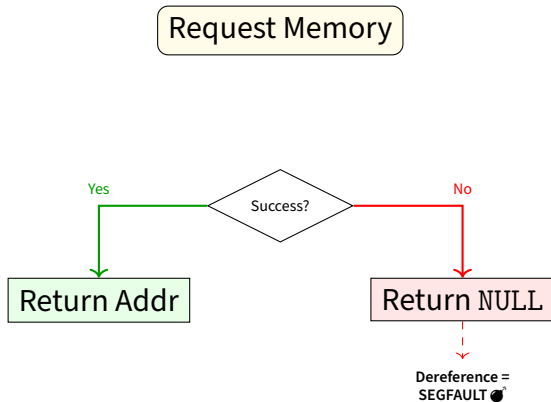
This is how you make an array that grows!

```
1 int *arr = malloc(2 * sizeof(int)); // Start small
2 int capacity = 2;
3 int count = 0;
4
5 while (input) {
6     if (count == capacity) {
7         capacity *= 2;
8         // Resize!
9         int *temp = realloc(arr, capacity * sizeof(int));
10        if (!temp) { /* Handle error */ }
11        arr = temp;
12    }
13    arr[count] = input;
14    count++;
15 }
16 free(arr);
17
```

The Golden Rule: Always Check `malloc`

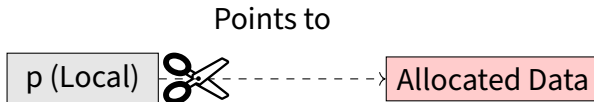
`malloc` is a **request**, not a command. The OS can say "No" (e.g., out of memory).

```
1 int *p = malloc(1000);  
2  
3 // "If p is NOT valid..."  
4 if (!p)  
5 {  
6     // Handle error  
7     gracefully  
8 }  
9 // Safe to use now  
10 *p = 10;  
11
```



Visualizing a Memory Leak

```
1 void leak()  
2 {  
3     int *p = malloc(1024);  
4     // Function ends, 'p' (Stack) is destroyed  
5     // Heap block remains! No one points to it!  
6 }  
7
```



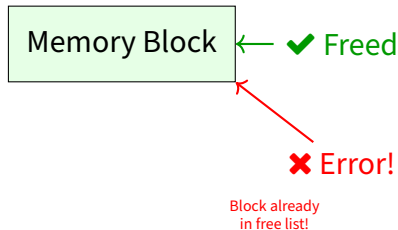
Lost forever (until reboot)

Pitfall 1: Double Free

Rule

You must `free()` every allocation exactly **once**.

```
1 int *p = malloc(sizeof(int));  
2  
3 free(p); // OK.  
4         // Memory marked as "  
5         Available"  
6 free(p); // CRASH! (Aborted)  
7         // Corrupts heap  
8         metadata.
```



Pitfall 2: Use After Free (Dangling Pointer)

After calling `free(p)`, the pointer `p` still holds the address, but the memory is gone!

The Bug:

```
1 char *s = malloc(10);
2 strcpy(s, "Hello");
3
4 free(s);
5 // s is now a "Dangling Pointer"
6
7 printf("%s", s);
8 // Undefined Behavior!
9 // Might print garbage, might crash.
```

The Fix:

```
1 free(s);
2 s = NULL; // Safety!
```

ptr (0x500) 

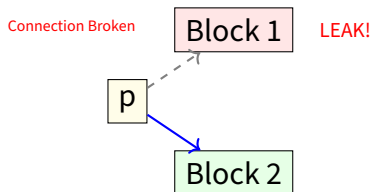
0x500

Belongs to OS now!
(Landmines)

Pitfall 3: The Overwrite Leak

If you overwrite a pointer before freeing it, you lose the link to the heap memory forever.

```
1 int *p = malloc(64);  
2  
3 // OOPS! We just lost the  
4 // address of the first block.  
5 p = malloc(64);  
6  
7 // We can only free the 2nd  
8   one.  
9 free(p);
```



Avoiding Memory Leaks



Best Practices:

- Always free dynamically allocated memory.
- Set freed pointers to NULL to avoid using dangling pointers.
- Use memory analysis tools like `valgrind` to detect leaks.

Avoiding Memory Leaks

```
1 #include <stdlib.h>
2 #include <stdio.h>
3
4 int main()
5 {
6     int *ptr = (int *)malloc(10 * sizeof(int));
7     if (!ptr)
8     {
9         return 1;
10    }
11    free(ptr); // Correctly freeing memory
12    ptr = NULL; // Prevents dangling pointer issues
13    return 0;
14 }
```

Helpful Debugging Tools

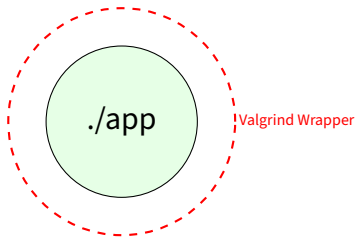
Tool 1: Valgrind (The Memory Police)

Valgrind is a tool that runs your program and watches every single memory access. It detects:

- **Memory Leaks** (Forgot free)
- **Invalid Reads/Writes** (Out of bounds)
- **Uninitialized Variables**

Usage:

```
1 // Compile with -g for line
  numbers!
2 gcc -g -Wall main.c -o app
3
4 // Run with valgrind
5 valgrind ./app
```



Runs slower,
but catches errors!

Reading Valgrind Output

Don't be scared by the wall of text! Look for the **Summary**.

```
1 ==12345== HEAP SUMMARY:
2 ==12345==      in use at exit: 4 bytes in 1 blocks
3 ==12345==    total heap usage: 1 allocs, 0 frees, 4 bytes allocated
4 ==12345==
5 ==12345== LEAK SUMMARY:
6 ==12345==    definitely lost: 4 bytes in 1 blocks
7 ==12345==    indirectly lost: 0 bytes in 0 blocks
8
```

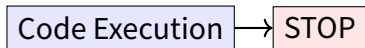
Definitely Lost → = You forgot to free() something!

Tool 2: GDB (The Detective)

GDB allows you to pause your program and inspect variables while it runs.

Scenario A: Logic Errors (No Malloc) *"Why is my loop going to infinity?"*

- 1 `gcc -g main.c`
- 2 `gdb ./a.out`
- 3 `(gdb) break main`
- 4 `(gdb) run`
- 5 `(gdb) next (Step over)`
- 6 `(gdb) print i (Check vars)`



Inspect variables
at this exact moment

GDB & Dynamic Memory (The Crash)

Scenario B: Segmentation Faults *"My program crashed! Where?"*

When using `malloc`, crashes often happen deep inside functions. GDB tells you exactly where.

```
1 // 1. Run until crash
2 (gdb) run
3 ...
4 Program received signal SIGSEGV,
5 Segmentation fault.
6 0x000000... in main () at main.c:10
7 10      *ptr = 42;
8
9 // 2. See "Who called who"
10 (gdb) backtrace
11 #0 func_c () at main.c:10
12 #1 func_b () at main.c:25
13 #2 main ()   at main.c:50
14
```



Segfault

Use **backtrace** (bt)
to trace the path
of destruction!

More information in the Kickoff Sessions! ;)

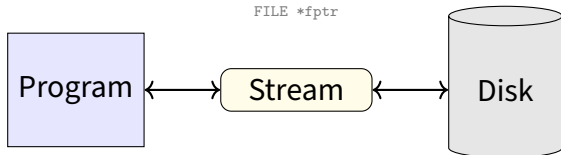
File I/O

File I/O: The Stream Abstraction

In C, files are accessed via **Streams**. A stream is a buffered connection between your program and a file on the disk.

The FILE Handle:

```
1 #include <stdio.h>
2
3 int main()
4 {
5     FILE *fptr;
6
7     fptr = fopen("data.txt",
8                 "r");
9 }
10
```



Opening Modes

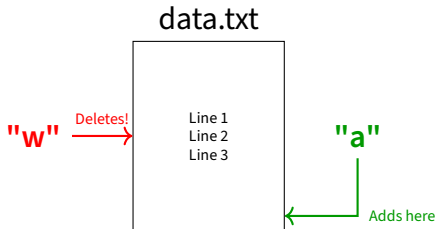
`fopen(filename, mode)` determines how you interact with the file.

Mode	Behavior
------	----------

"r"	Read: Fails if file doesn't exist.
-----	---

"w"	Write: WIPES file content!
-----	--

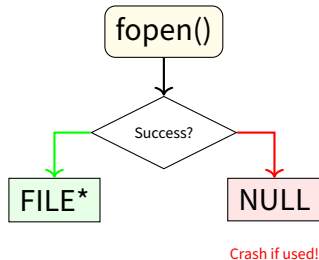
"a"	Append: Adds to the end.
-----	---------------------------------



Step 2: Always Check for NULL

`fopen` returns `NULL` if the file is missing or locked.

```
1 FILE *f = fopen("missing.txt", "r");
2
3 if (f == NULL)
4 {
5     printf("Error: File not found!\n");
6     return 1; // Exit cleanly
7 }
8
9 // Only safe if f != NULL
10 fscanf(f, ...);
11
```



Step 3: Writing Data (fprintf)

Writing to a file is identical to printing to the console, just add the file pointer.

```
1 int score = 100;
2 char *name = "Player1";
3
4 // To Console
5 printf("User: %s, Score: %d\n",
6       name, score);
7
8 // To File
9 fprintf(fp, "User: %s, Score: %d\n",
10        name, score);
11
```

fprintf(...)

User: Player1,
Score: 100

file.txt

Step 4: Reading & Closing

Reading: `fscanf` reads formatted input (like `scanf`). **Closing:** `fclose` is mandatory to save data!

```
1 int num;  
2 // Reads one integer from file  
3 fscanf(fp_ptr, "%d", &num);  
4  
5 // MANDATORY: Closes stream & saves changes  
6 fclose(fp_ptr);  
7
```

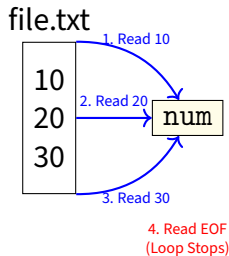


Flushes Buffer
(Saves Data)

Reading a Whole File (The Loop)

`fscanf` returns the number of items successfully read. It returns EOF (End Of File) when done.

```
1 int num;  
2  
3 // Loop while we successfully  
4 // read 1 integer until we hit EOF  
5 while (fscanf(f, "%d", &num) == 1)  
6 {  
7     printf("Read: %d\n", num);  
8 }  
9 printf("Done!");  
10
```



Reading Whole Lines (fgets)

fscanf stops at the first space. To read a full sentence, use fgets.

Input File: Hello World

1. fscanf (Stops early)

```
1 char buf[100];  
2 fscanf(f, "%s", buf);  
3 // buf = "Hello" (Lost "World")  
4
```

fscanf

"Hello"

2. fgets (Reads Line)

```
1 char buf[100];  
2 fgets(buf, 100, f);  
3 // buf = "Hello World\n"  
4
```

fgets

"Hello World\n"



Reading Formatted Data (CSV)

If you know the file structure (e.g., Name , Age), you can format the scan string.

File Content: Alice,20

```
1 char name[50];
2 int age;
3
4 // Note the comma in the format string!
5 // It tells C to expect a comma in the file.
6 if (fscanf(f, "%[^,],%d", name, &age) == 2)
7 {
8     printf("User: %s is %d years old.\n", name, age);
9 }
10
```

Writing Lists to Files

Common Task: Saving an array of data to a log file.

```
1 int scores[] = {10, 55, 99};
2 FILE *f = fopen("log.txt", "w");
3
4 if (f) // Same as if (f != NULL)
5 {
6     fprintf(f, "--- RESULTS ---\n");
7     for (int i = 0; i < 3; i++)
8     {
9         fprintf(f, "ID: %d | Score: %d\n",
10                i, scores[i]);
11     }
12     fclose(f);
13 }
```

log.txt

```
-- RESULTS --
ID: 0 | Score: 10
ID: 1 | Score: 55
ID: 2 | Score: 99
```

Questions?

Introduction to Structured Programming KU

Session 3: Structs & Dynamic Memory

Sebastian David Albu

Summer Semester 2026

Institute of Software Engineering and Artificial Intelligence

LaTeX Template provided by the Institute of Information Security